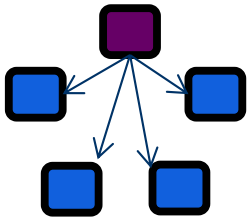
CompSci 390-03-26S

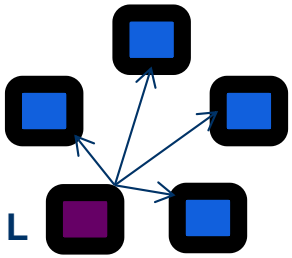
Distributed and Networked Systems

Raft Consensus



State Machine Replication (SMR/RSM)

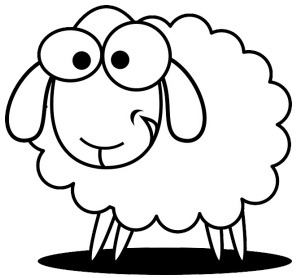
- N replicas: N = 2f+1 to mask up to f faults—highly available, "always on".
- Replicas run the same app program P with the same API. It doesn't matter what P is (reusable!), but P must be **deterministic**.
  - P is a **state machine**: function {state, action}→state
- Each replica keeps its own copy of internal app state for P.
- Suppose replicas agree on a sequence (or log) of actions to apply.
  - Ordered list of (requests, commands, ops). **Linearizable**.
- Then they converge.
  - S/elect stable leader
  - Leader handles client RPCs
  - Others follow along as **backups**



The players

Leader/Primary

1. Become leader.
2. Rewrite history.
3. Dictate the future.
4. Stay leader forever.
5. If deposed goto step 1.

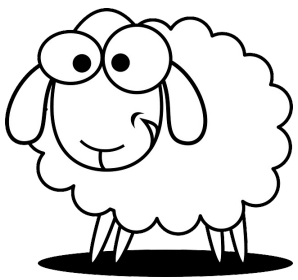


Acceptor/Replica/**Follower**

1. Adopt leader.
2. Tell it your history.
3. Accept whatever the leader says.
4. Write it all down.
5. If a new leader appears, goto step 1.

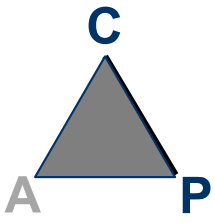
Consensus oversimplified

1. If leader falters try to take over.
2. Become leader by majority vote/assent.
3. Discover and affirm history.
4. Propose values for new log slots in order; notify others if majority accepts.



1. Adopt leader.
2. Tell it your history.
3. Accept whatever the leader says.
4. Write it all down.
5. If a new leader appears, goto step 1.

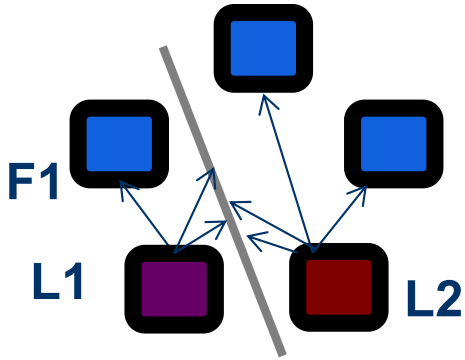
Consensus: Quorum rule



In order to serve and commit, a leader L1 must continually receive votes from a **majority** or **quorum**: f+1 of the full 2f+1 group.

- **Safe** (C). In a partition scenario, at most one subgroup can serve requests.
- But not **live** (A). If a majority of replicas fail, it might be safe for the survivors to serve. But they **must not** because they cannot rule out a partition.

L1 and F1 do not have a quorum. So they must not serve clients → no **A**vailability if the majority had merely failed.

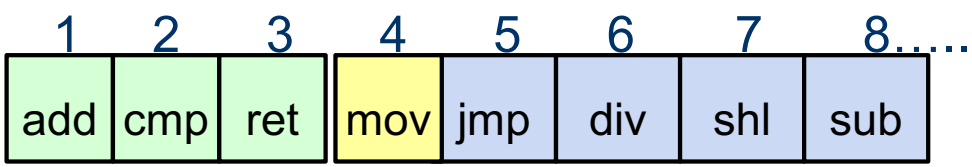


L2 can gain a quorum from the other followers, so its side of the partition can make progress.

At most one side of a partition makes progress because **disjoint majorities cannot exist**.

The operation log

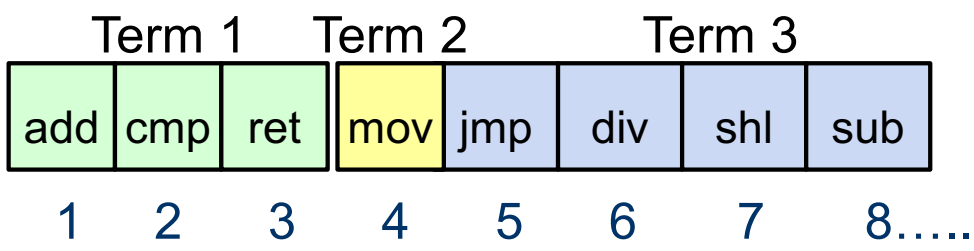
- Each replica has a local log of accepted actions (ops, commands).
- An operation log is a sequence of consecutively numbered entries.
  - Raft: log entry / log **index**. (Paxos: slots, VR: op-numbers)
  - Each entry has a corresponding action/op/command.
- Goals:
  - Agree on an action/op/command for each index (consensus).
  - Recognize when it is safe to **commit**: extend the committed log prefix.



Graphic from Raft authors

Terms/views

- The consensus protocol runs as a sequence of **terms**.
  - VR calls them **views**
- Terms are numbered: monotonically increasing.
  - Any replica can declare a new term on suspicion the leader has failed.
- Every message and log entry is tagged with a term#.
  - (term, index) pair is called a **viewstamp** in VR
  - "Number go up"



Graphic from Raft authors

State

Persistent state on all servers:
(Updated on stable storage before responding to RPCs)

currentTerm latest term server has seen (initialized to 0 on first boot, increases monotonically)

votedFor candidateId that received vote in current term (or null if none)

log[] log entries; each entry contains command for state machine, and term when entry was received by leader (first index is 1)

Volatile state on all servers:

commitIndex index of highest log entry known to be committed (initialized to 0, increases monotonically)

lastApplied index of highest log entry applied to state machine (initialized to 0, increases monotonically)

Volatile state on leaders:
(Reinitialized after election)

nextIndex[] for each server, index of the next log entry to send to that server (initialized to leader last log index + 1)

matchIndex[] for each server, index of highest log entry known to be replicated on server (initialized to 0, increases monotonically)

AppendEntries RPC

Invoked by leader to replicate log entries (§5.3); also used as heartbeat (§5.2).

Arguments:

term leader's term

leaderId so follower can redirect clients

prevLogIndex index of log entry immediately preceding new ones

prevLogTerm term of prevLogIndex entry

entries[] log entries to store (empty for heartbeat; may send more than one for efficiency)

leaderCommit leader's commitIndex

Results:

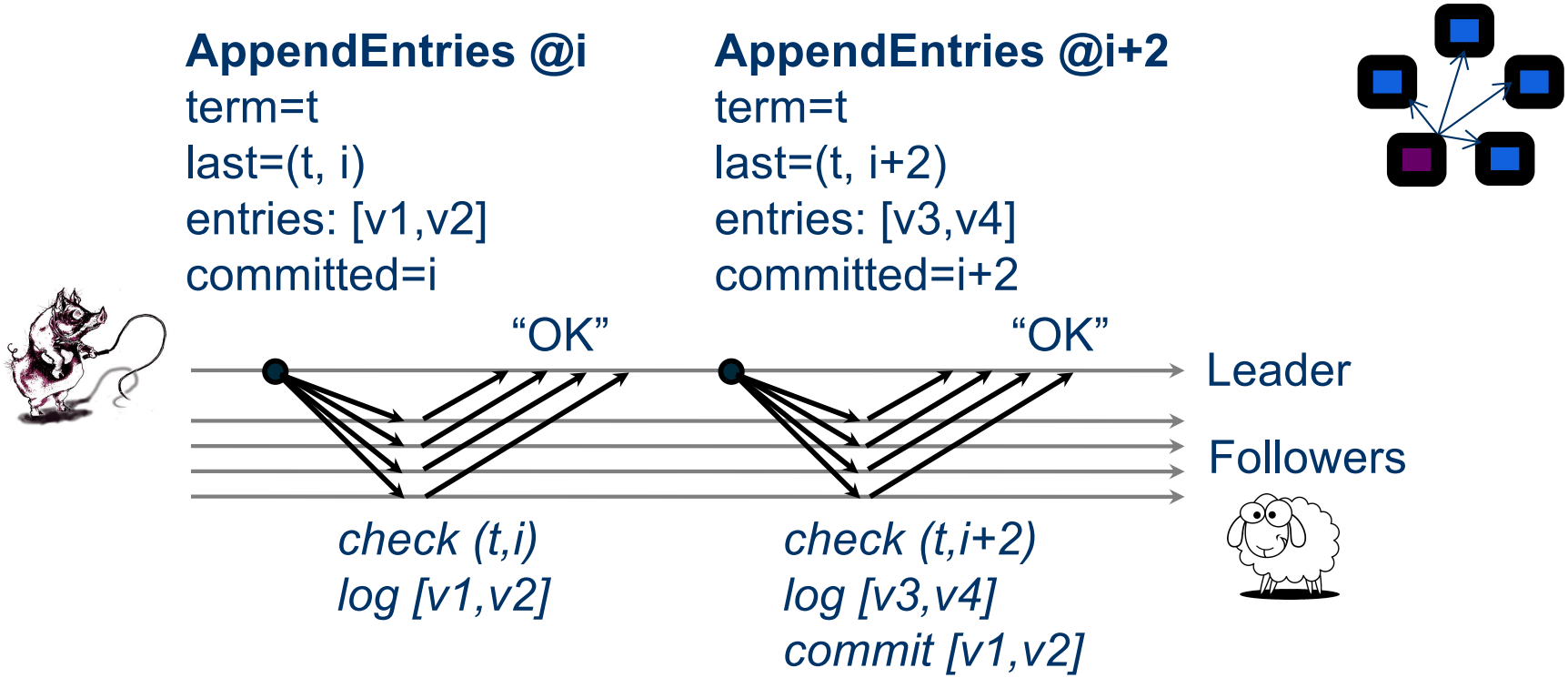
term currentTerm, for leader to update itself

success true if follower contained entry matching prevLogIndex and prevLogTerm

Receiver implementation:

1. Reply false if term < currentTerm (§5.1)
2. Reply false if log doesn't contain an entry at prevLogIndex whose term matches prevLogTerm (§5.3)
3. If an existing entry conflicts with a new one (same index but different terms), delete the existing entry and all that follow it (§5.3)
4. Append any new entries not already in the log
5. If leaderCommit > commitIndex, set commitIndex = min(leaderCommit, index of last new entry)

Raft in normal operation (stable term)

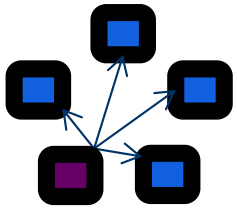


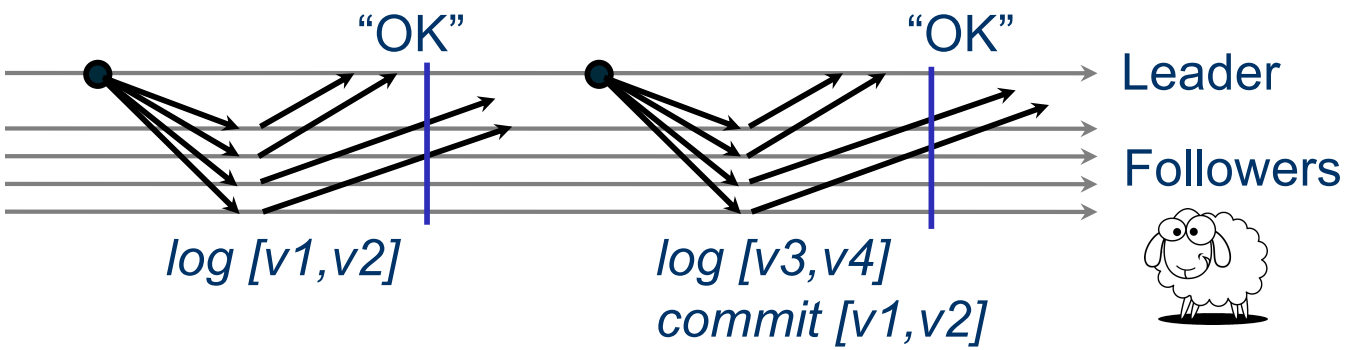
Followers accept what their leader tells them, even if it rewrites history. But first: check that leader's last viewstamp (term, index) is "in sync".

Commitment

AppendEntries @i
entries: [v1,v2]
committed=i

AppendEntries @i+2
entries: [v3,v4]
committed=i+2





Note that **log** here means to add a record to the replicated log (in memory) and commit means the committed entry will never go back.

What if replicas lag? **When is it safe to commit?**

Commit wave: let's do this

This Raft pic shows how the logs advance in a wavefront.

At any given time the replicas agree on a common log prefix.

This figure applies to other Consensus variants.

Index ←→ slot ←→ op-number
Term ←→ ballot ←→ view

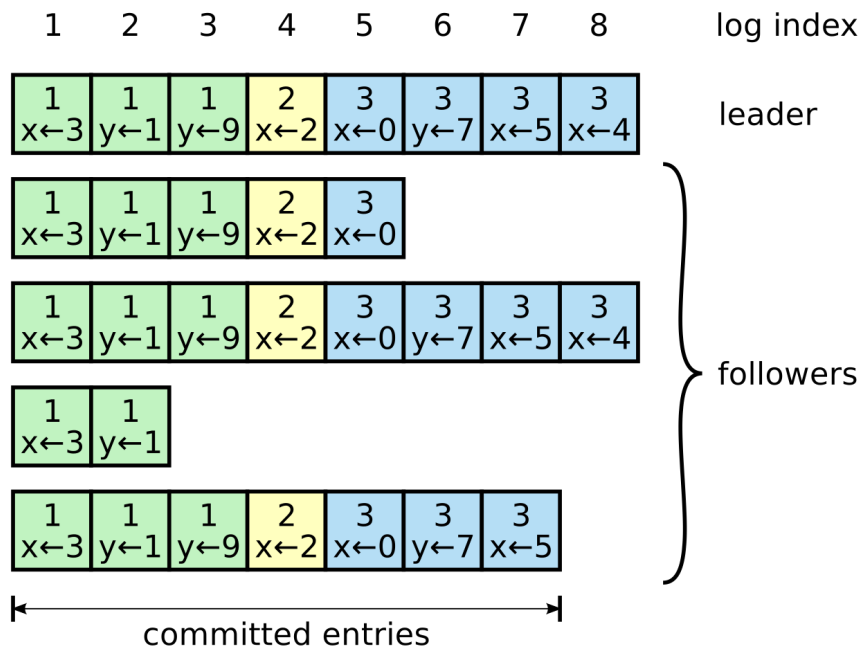


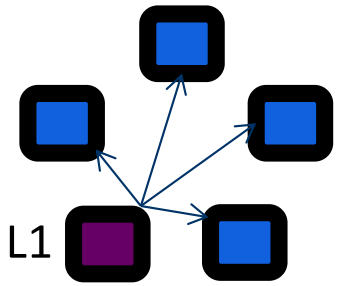
Figure 6: Logs are composed of entries, which are numbered sequentially. Each entry contains the term in which it was created (the number in each box) and a command for the state machine. An entry is considered *committed* if it is safe for that entry to be applied to state machines.

Graphic from Raft paper

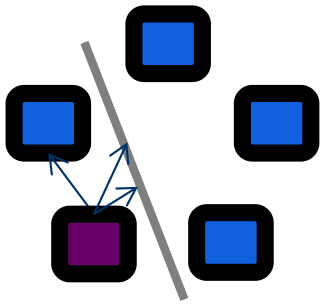
But partitions! A nasty scenario…

Consider a network partition: L1 survives. Split brain?

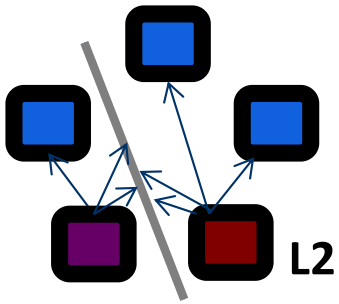
- How to make progress where we can do it safely?
- How to block progress where it is unsafe?



A happy group of 5 replicas.



Oops, partition.



Majority side elects L2; L1 struggles on.

Replica mode transitions

- Replicas keep their current role in a mode state variable.
- Haven't heard from your leader for "too long"? Timeout!
- Timeout: initiate leader election, declare new term++.
- Quorum → at most one leader can commit per view/term.

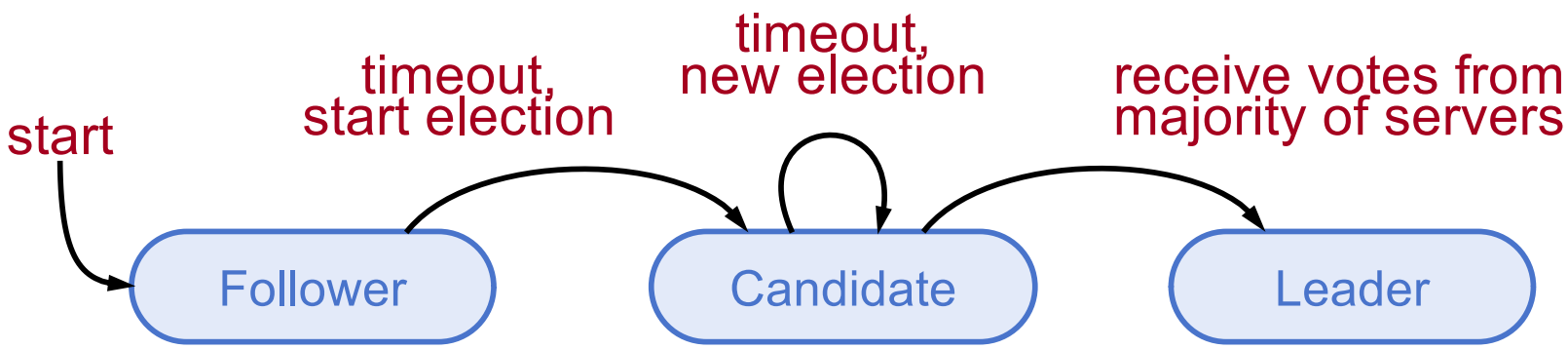


Image from authors of Raft Consensus Algorithm

RequestVote RPC

Invoked by candidates to gather votes (§5.2).

Arguments:

term	candidate's term
candidateId	candidate requesting vote
lastLogIndex	index of candidate's last log entry (§5.4)
lastLogTerm	term of candidate's last log entry (§5.4)

viewstamp

Results:

term	currentTerm, for candidate to update itself
voteGranted	true means candidate received vote

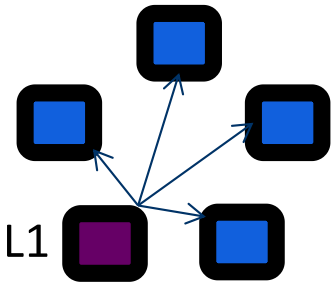
Receiver implementation:

1. Reply false if term < currentTerm (§5.1)
2. If votedFor is null or candidateId, and candidate's log is at least as up-to-date as receiver's log, grant vote (§5.2, §5.4)

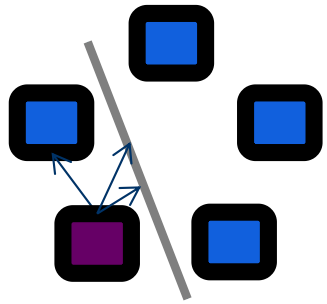
A nasty scenario continued: network heals

Partition heals → L1 and L2 both try to lead.

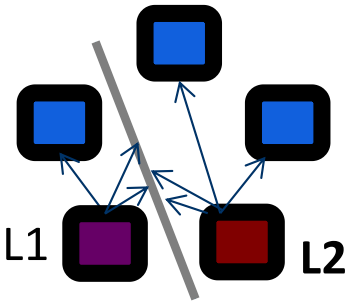
- How to establish which leader is "current"?
- How to reconcile/repair conflicting histories in the logs?
- How to ensure all committed entries survive?



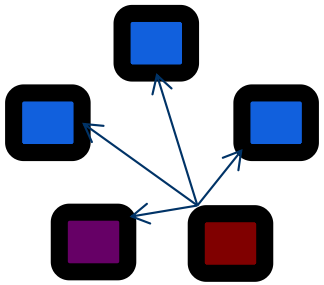
A happy group of 5 replicas.



Oops, partition.



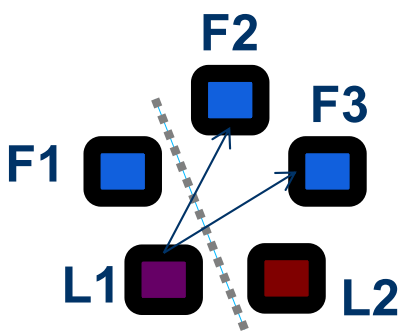
Majority side elects L2; L1 struggles on.



Partition heals. **Two leaders?**

Handling stale views/terms

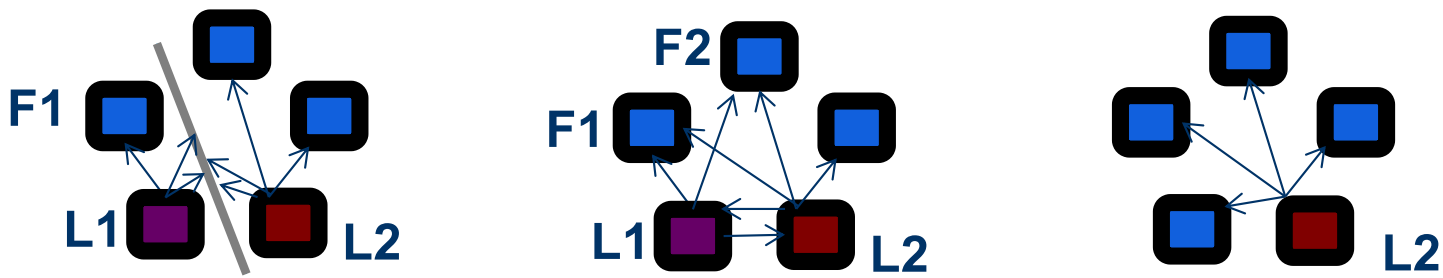
- Every message in the protocol carries the sender's term.
- If the receiver's term is higher, it ignores/rejects the message.
  - Stale participants can do no damage.
- If a participant learns of a term/view v higher than its own (stale!), it sets its term to v…
- (Yes, the term/view is an example of a **logical clock**.)



The net heals, and L1 reappears suddenly! But F2 and F3 are following a new leader L2 now. They say to L1: "I won't follow you."

Rejecting a leader from the past

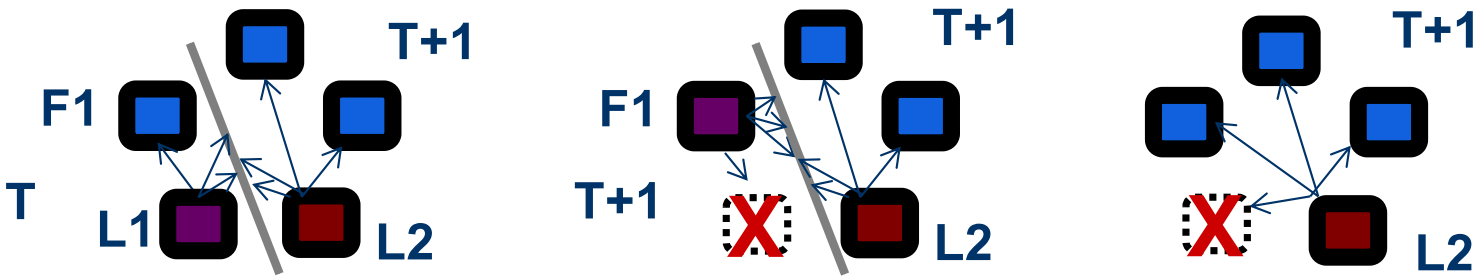
- Replicas that hear L2 adopt L2's term—the latest and greatest.
- Those replicas ignore old leader L1: its term has expired—sad!
- L1 steps down when it learns of a term later than its own.



During the partition, L2 declares candidacy in a new term and wins a quorum. When the partition heals, F1 hears L2 is the new leader in town, and defects to L2. F2 ignores L1, whose term has expired. L1 learns of the new term and steps down.

Never two kings at once

- Multiple replicas may attempt to lead with a term T, but at most one can win a quorum to lead in T; the loser(s) **never** create entries with term T.
- Therefore, if two replicas have divergent entries at log index i, then the entries to keep (e.g., committed by new leader) **always** have a higher term.



L1 leads in term T, then: partition! And then L1 fails! F1 and L2 declare candidacy with term T+1. But only L2 wins a quorum. F1's candidacy is doomed: it never becomes leader, and so never accepts a client request, and never creates a log entry with term T+1. When the partition heals, F1 hears and follows L2 in T+1.

Replica mode transitions

- Replicas transition state upon hearing of a new term.
- Even the followers defect to a new leader in town.

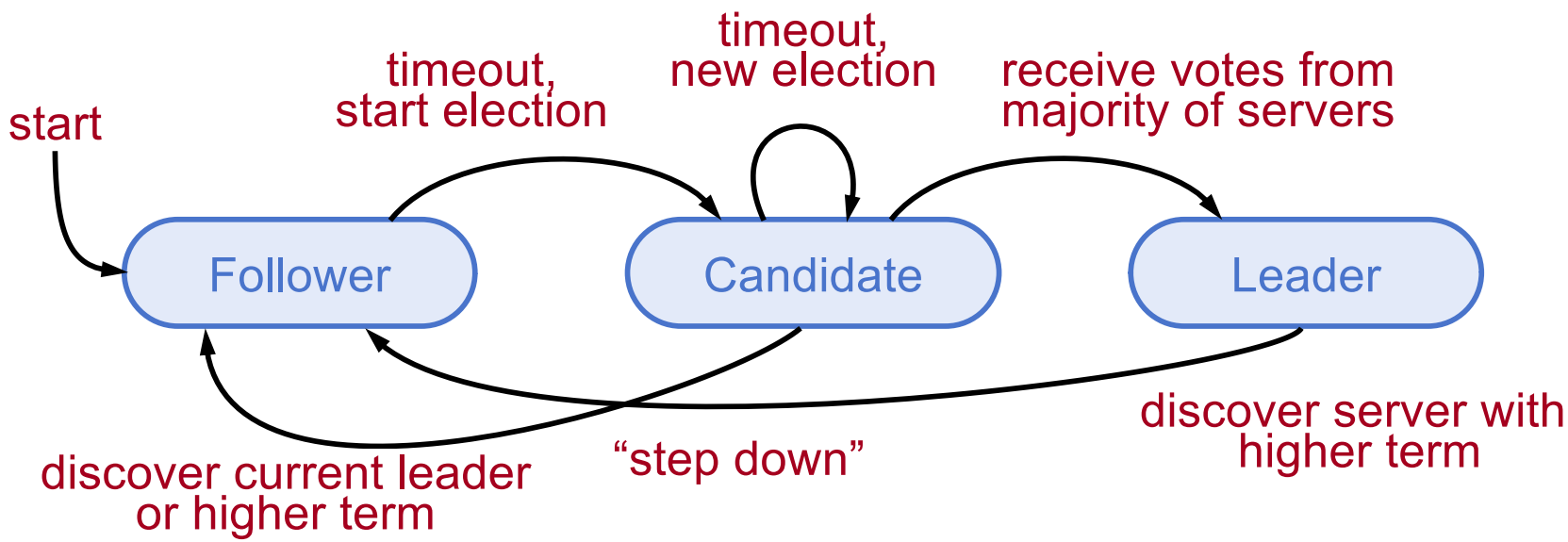
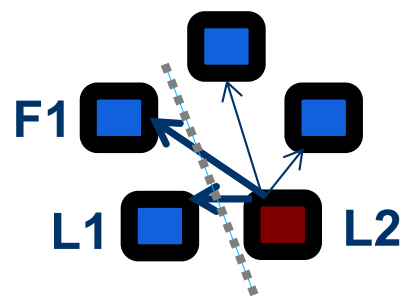


Image from authors of Raft Consensus Algorithm

Catching up

- After the network heals, L2 adopts new followers: F1 and L1.
- The new followers reject L2's append calls: "catch me up".
- L2 uses **repair-**appends to overwrite their log tails with its own.
- Visible to clients?

"Catch me up!"



The replicas regroup with L2 continuing as leader after the partition heals. L2 discovers that the nodes isolated on the minority side lag behind. L2 appends its log tail onto the new followers, starting from the common log prefix.

State

Persistent state on all servers:
(Updated on stable storage before responding to RPCs)

currentTerm latest term server has seen (initialized to 0 on first boot, increases monotonically)

votedFor candidateId that received vote in current term (or null if none)

log[] log entries; each entry contains command for state machine, and term when entry was received by leader (first index is 1)

Volatile state on all servers:

commitIndex index of highest log entry known to be committed (initialized to 0, increases monotonically)

lastApplied index of highest log entry applied to state machine (initialized to 0, increases monotonically)

Volatile state on leaders:
(Reinitialized after election)

nextIndex[] for each server, index of the next log entry to send to that server (initialized to leader last log index + 1)

matchIndex[] for each server, index of highest log entry known to be replicated on server (initialized to 0, increases monotonically)

AppendEntries RPC

Invoked by leader to replicate log entries (§5.3); also used as heartbeat (§5.2).

Arguments:

term leader's term

leaderId so follower can redirect clients

prevLogIndex index of log entry immediately preceding new ones

prevLogTerm term of prevLogIndex entry

entries[] log entries to store (empty for heartbeat; may send more than one for efficiency)

leaderCommit leader's commitIndex

Results:

term currentTerm, for leader to update itself

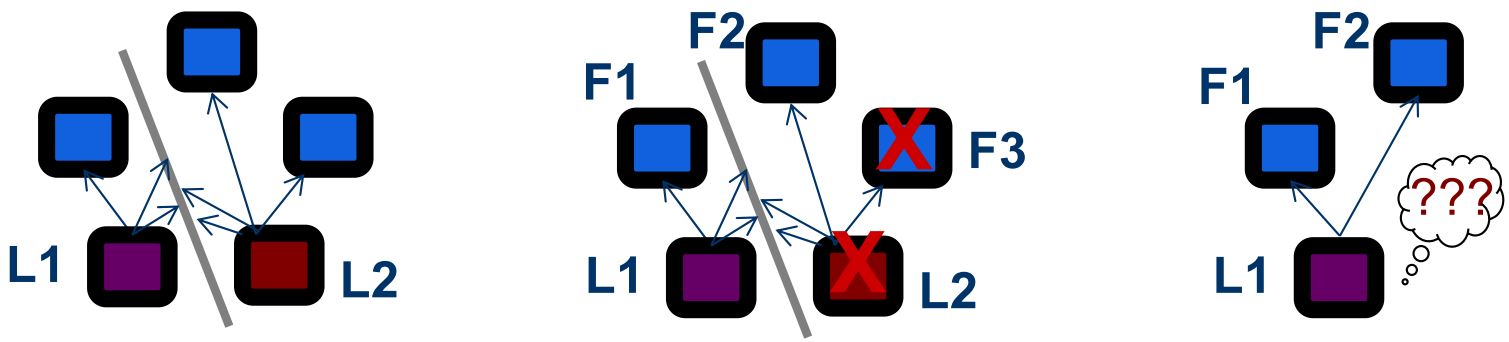
success true if follower contained entry matching prevLogIndex and prevLogTerm

Receiver implementation:

1. Reply false if term < currentTerm (§5.1)
2. Reply false if log doesn't contain an entry at prevLogIndex whose term matches prevLogTerm (§5.3)
3. If an existing entry conflicts with a new one (same index but different terms), delete the existing entry and all that follow it (§5.3)
4. Append any new entries not already in the log
5. If leaderCommit > commitIndex, set commitIndex = min(leaderCommit, index of last new entry)

Nasty scenario: still nastier

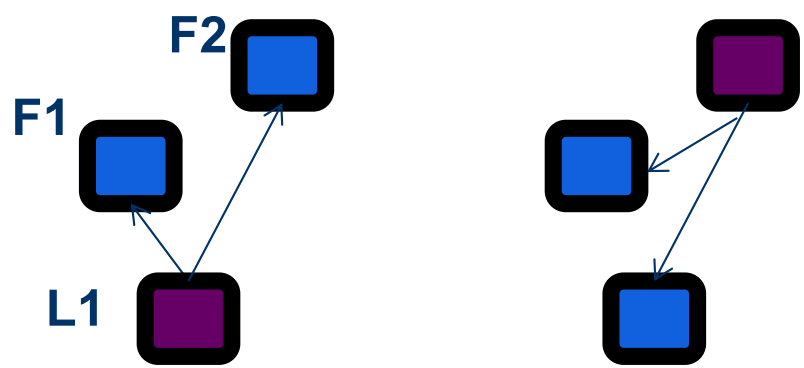
- **Suppose** as before: a bare majority commit new entries under L2.
- The minority writes new uncommitted entries under L1.
- The net heals and the minority rejoins, **but then L2 and a follower (F3) fail**!
- The survivors can make progress—if they can patch up their differences.
- Who knows the "right" committed history? How to propagate it?



Nasty scenario: still nastier

- Log repair considers the term at each log index (the **viewstamps**).
- As before: F2 rejects L1 and L1 steps down. (F2's term/viewstamp wins.)
- A new election is declared → new term with a new leader. **But who**?
- F2 has log entries from a higher term than the others. Higher terms/viewstamps dominate → the others accept F2's entries.
- **In Raft**, F2 declines to vote for a lagging candidate → becomes leader itself.

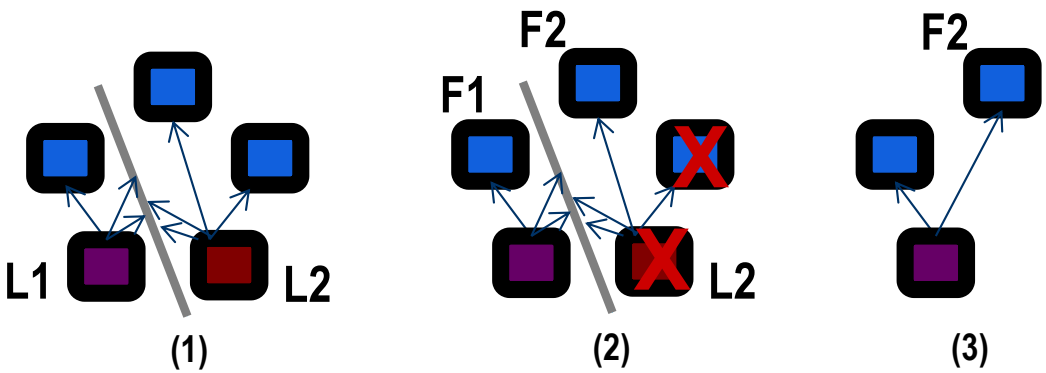
F2 rejects L1.
L1 steps down.
A new term begins.
An election starts.



Suppose F2 wins. (In Raft, it will). As leader, it pushes its log onto the others, rewriting their history.

Safety: committed entries survive into future views

- In our scenario, one replica (F2) survives to propagate commits into a new term.



- **Provable**: for **any** committed log entry *e*, **at least one** replica who knows of *e* survives into any future term (that has a new leader), always.
- Why? It requires a majority quorum to commit entries or elect a new leader, and any two such quorums (quora?) **must** intersect.

Consensus design choices

Raft picks an up-to-date replica as leader

If log entries prepare/decide sequentially (as in Raft, VR, and Multi-Paxos) then a replica's viewstamp captures how current that replica is.

- Viewstamp is: (highest term, highest index for term)
- Within any voting majority, we can identify a replica R whose viewstamp is at least as up to date as all of the others.
- In VR, new leader is round robin but adopts the log of R in its entirety.
- In Multi-Paxos…well, go read it and see if you can figure out what it does.
- **In Raft, only such an "all caught up" R can win a vote for leader.**
  - A replica V votes for candidate C only if C is at least as up to date as V.
  - No need to transfer state to a new leader→one-way state transfer.

RequestVote RPC

Invoked by candidates to gather votes (§5.2).

Arguments:

term	candidate's term
candidateId	candidate requesting vote
lastLogIndex	index of candidate's last log entry (§5.4)
lastLogTerm	term of candidate's last log entry (§5.4)

viewstamp

Results:

term	currentTerm, for candidate to update itself
voteGranted	true means candidate received vote

Receiver implementation:

1. Reply false if term < currentTerm (§5.1)
2. If votedFor is null or candidateId, and candidate's log is at least as up-to-date as receiver's log, grant vote (§5.2, §5.4)

Raft: safety

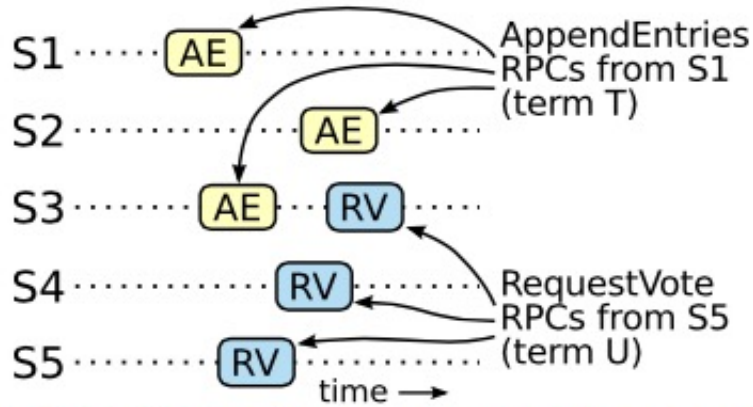


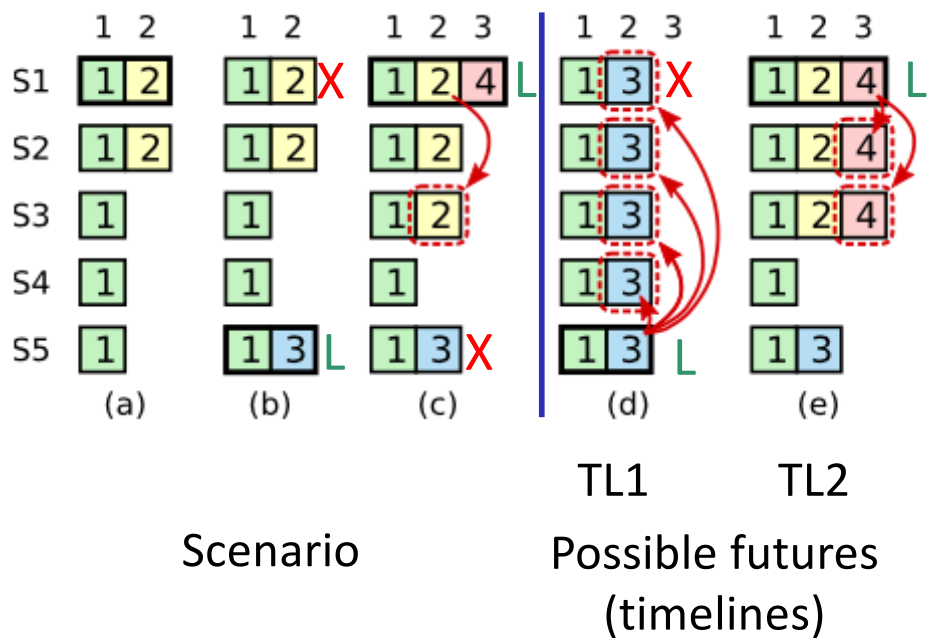
Figure 9: If S1 (leader for term T) commits a new log entry from its term, and S5 is elected leader for a later term U, then there must be at least one server (S3) that accepted the log entry and also voted for S5.

Graphic from Raft paper

Note: when a new leader L propagates log entries into a new term, L does not and cannot (always) know if they committed or not in an older term. But it does not matter: the set of surviving entries is guaranteed to include all committed entries, whether known or not. And once L propagates them to a new quorum, they are known to have committed.

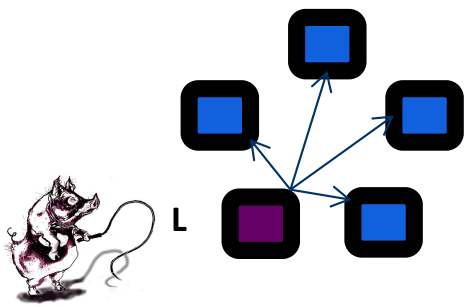
Committed? Who knows?

Figure 8: A time sequence showing why a leader cannot determine commitment using log entries from older terms. In (a) S1 is leader and partially replicates the log entry at index 2. In (b) S1 crashes; S5 is elected leader for term 3 with votes from S3, S4, and itself, and accepts a different entry at log index 2. In (c) S5 crashes; S1 restarts, is elected leader, and continues replication. At this point, the log entry from term 2 has been replicated on a majority of the servers, but it is not committed. If S1 crashes as in (d), S5 could be elected leader (with votes from S2, S3, and S4) and overwrite the entry with its own entry from term 3. However, if S1 replicates an entry from its current term on a majority of the servers before crashing, as in (e), then this entry is committed (S5 cannot win an election). At this point all preceding entries in the log are committed as well.

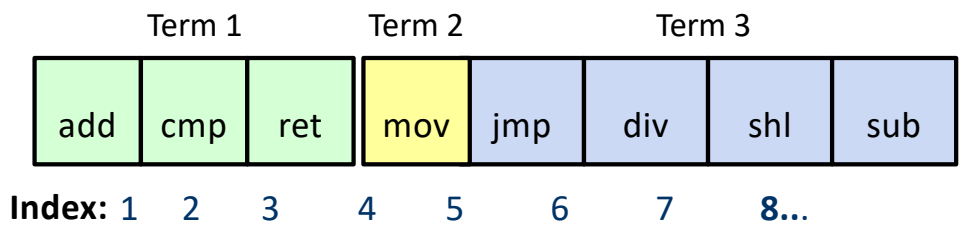


Graphic from Raft paper, annotated

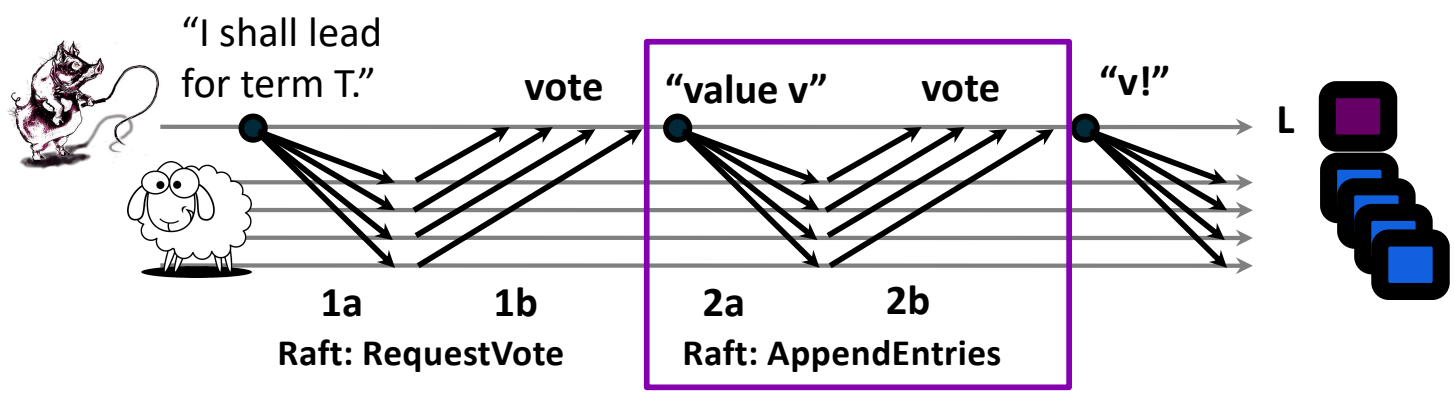
Consensus: review



A happy group of 5 replicas.



Each has a copy of a log of (ballot, value) pairs.
Term (ballot, view), **index** (slot)**; op** (command, action).



Summary

- Leader/master coordinates, dictates consensus
  - View the service as a deterministic state machine.
  - Master (also called "primary") dictates order of client operations.
  - → All non-faulty replicas reach the same state.
- Remaining problem: who is the leader?
  - Leader itself might fail or be isolated by a network partition.
  - Requires a scheme for "leader election" to choose a new leader.
  - Consensus is safe but not live: in the worst case (multiple repeated failures) it might not terminate.
  - But in practice Consensus gets the job done…if it can be done.

State

Persistent state on all servers:
(Updated on stable storage before responding to RPCs)

currentTerm	latest term server has seen (initialized to 0 on first boot, increases monotonically)
votedFor	candidateId that received vote in current term (or null if none)
log[]	log entries; each entry contains command for state machine, and term when entry was received by leader (first index is 1)

Volatile state on all servers:

commitIndex	index of highest log entry known to be committed (initialized to 0, increases monotonically)
lastApplied	index of highest log entry applied to state machine (initialized to 0, increases monotonically)

Volatile state on leaders:
(Reinitialized after election)

nextIndex[]	for each server, index of the next log entry to send to that server (initialized to leader last log index + 1)
matchIndex[]	for each server, index of highest log entry known to be replicated on server (initialized to 0, increases monotonically)

AppendEntries RPC

Invoked by leader to replicate log entries (§5.3); also used as heartbeat (§5.2).

Arguments:

term	leader's term
leaderId	so follower can redirect clients
prevLogIndex	index of log entry immediately preceding new ones
prevLogTerm	term of prevLogIndex entry
entries[]	log entries to store (empty for heartbeat; may send more than one for efficiency)
leaderCommit	leader's commitIndex

Results:

term	currentTerm, for leader to update itself
success	true if follower contained entry matching prevLogIndex and prevLogTerm

Receiver implementation:

1. Reply false if term < currentTerm (§5.1)
2. Reply false if log doesn't contain an entry at prevLogIndex whose term matches prevLogTerm (§5.3)
3. If an existing entry conflicts with a new one (same index but different terms), delete the existing entry and all that follow it (§5.3)
4. Append any new entries not already in the log
5. If leaderCommit > commitIndex, set commitIndex = min(leaderCommit, index of last new entry)

RequestVote RPC

Invoked by candidates to gather votes (§5.2).

Arguments:

term	candidate's term
candidateId	candidate requesting vote
lastLogIndex	index of candidate's last log entry (§5.4)
lastLogTerm	term of candidate's last log entry (§5.4)

Results:

term	currentTerm, for candidate to update itself
voteGranted	true means candidate received vote

Receiver implementation:

1. Reply false if term < currentTerm (§5.1)
2. If votedFor is null or candidateId, and candidate's log is at least as up-to-date as receiver's log, grant vote (§5.2, §5.4)

Rules for Servers

All Servers:
- If commitIndex > lastApplied: increment lastApplied, apply log[lastApplied] to state machine (§5.3)
- If RPC request or response contains term T > currentTerm: set currentTerm = T, convert to follower (§5.1)

Followers (§5.2):
- Respond to RPCs from candidates and leaders
- If election timeout elapses without receiving AppendEntries RPC from current leader or granting vote to candidate: convert to candidate

Candidates (§5.2):
- On conversion to candidate, start election:
  - Increment currentTerm
  - Vote for self
  - Reset election timer
  - Send RequestVote RPCs to all other servers
- If votes received from majority of servers: become leader
- If AppendEntries RPC received from new leader: convert to follower
- If election timeout elapses: start new election

Leaders:
- Upon election: send initial empty AppendEntries RPCs (heartbeat) to each server; repeat during idle periods to prevent election timeouts (§5.2)
- If command received from client: append entry to local log, respond after entry applied to state machine (§5.3)
- If last log index ≥ nextIndex for a follower: send AppendEntries RPC with log entries starting at nextIndex
  - If successful: update nextIndex and matchIndex for follower (§5.3)
  - If AppendEntries fails because of log inconsistency: decrement nextIndex and retry (§5.3)
- If there exists an N such that N > commitIndex, a majority of matchIndex[i] ≥ N, and log[N].term == currentTerm: set commitIndex = N (§5.3, §5.4).

Safety properties

Election Safety: at most one leader can be elected in a given term. §5.2

Leader Append-Only: a leader never overwrites or deletes entries in its log; it only appends new entries. §5.3

Log Matching: if two logs contain an entry with the same index and term, then the logs are identical in all entries up through the given index. §5.3

Leader Completeness: if a log entry is committed in a given term, then that entry will be present in the logs of the leaders for all higher-numbered terms. §5.4

State Machine Safety: if a server has applied a log entry at a given index to its state machine, no other server will ever apply a different log entry for the same index. §5.4.3

Figure 3: Raft guarantees that each of these properties is true at all times. The section numbers indicate where each property is discussed.

Consensus: stuff to know

- tolerate up to f failures; 2f+1 replicas → surviving f+1 is a quorum
- At most one viable leader at at any given time.
  - There might be multiple nodes in LEADER state, but at most one has a functioning quorum that can commit entries.
- Safety: the leader protects/preserves committed history.
  - Monotonic terms and quorum rules make it work, always.
- Election and commitment require a majority/quorum of the full configured group, not just of those present.
  - Quorum is key to preserving consistency/safety under partition.
- Not for scale. Adding replicas to an RSM group makes you do more work, but work doesn't get done any faster.